Thrust vector control using Arduino

Fazakas David-Antal

May 24, 2026

Contents

1 Objectives of the project **1**

2 Component description **1**
2.1 Actuators: MG90S Micro Servo . . . 2
2.2 Inertial Measurement Unit: MPU-6050 . . . 2
2.3 Microcontroller: ATmega328P . . . 3
2.4 Structural Components: 3D-Printed Gimbal Mount . . . 3

3 Description of work **5**
3.1 Proportional-Integral-Derivative (PID) Control Law . . . 5
3.2 System Limitations and Mechanical Challenges . . . 5
3.2.1 Manufacturing Tolerances and Actuation Limits . . . 5
3.2.2 Linkage Failure and Field Modification . . . 5
3.3 Electrical Schematic and Wiring Connections . . . 6
3.4 Setup architecture . . . 7
3.5 Pinout Table . . . 9

4 Working with registers **9**
4.1 General Purpose Input/Output (GPIO) Registers . . . 9
4.2 Timer0 Configuration: System Timing . . . 9
4.3 Timer1 Configuration: Fast PWM Generation . . . 10
4.3.1 Duty Cycle Calculus for Actuation . . . 10
4.4 Two-Wire Interface (TWI) Configuration . . . 10

5 Conclusions **10**

1 Objectives of the project

The aim of the project is to create a simple demonstration for a thrust vector control system which is used in the space industry to control the stability and direction of rockets. The cylindrical object(cardboard toilet paper roll) represents the gimbal direction of the rocket, while the board and the accelerometer are fixed onto a 3D printed stand, which represents the rocket proper.

2 Component description

The project uses as its main microcontroller an Arduino Uno board with the **ATmega328P** microcontroller on it. To the digital pins, two servos(**MG90S**) are connected, which use metal bearings to withstand the quick changes in spinning direction. The two servos are held in place on a gimbal stand using screws. The gimbal stand allows the "rocket gimbal", which is simulated by a toilet paper roll, to rotate in the ZX and ZY planes. Thus the servo motors can control the rocket's firing angle with respect to the rocket's ideal orientation and the orientation of the board, which represents the control system of the rocket and thus the orientation of the main rocket body.

2.1 Actuators: MG90S Micro Servo

To physically manipulate the Thrust Vector Control gimbal, two **MG90S** micro servos were utilized [1]. The MG90S was selected over standard plastic-geared servos (such as the SG90) due to its metal gear train, which provides the necessary mechanical durability to withstand the rapid, high-frequency directional changes demanded by an active PID control loop.

Operating at $4.8\,\text{V}$, the motor delivers a stall torque of $1.8\,\text{kg}\cdot\text{cm}$ and an operating speed of $0.10\,\text{s}/60^\circ$, ensuring adequate response time for attitude correction. The servos are driven via Pulse Width Modulation (PWM) at a standard $50\,\text{Hz}$ ($20\,\text{ms}$) refresh rate. However, to achieve the full 180° mechanical sweep, the control signal requires an extended pulse width range of $544\,\mu\text{s}$ to $2400\,\mu\text{s}$, deviating from the traditional $1000\,\mu\text{s}$ to $2000\,\mu\text{s}$ standard. This extended range was explicitly accounted for in the register level timer configurations to prevent control saturation and mechanical binding.

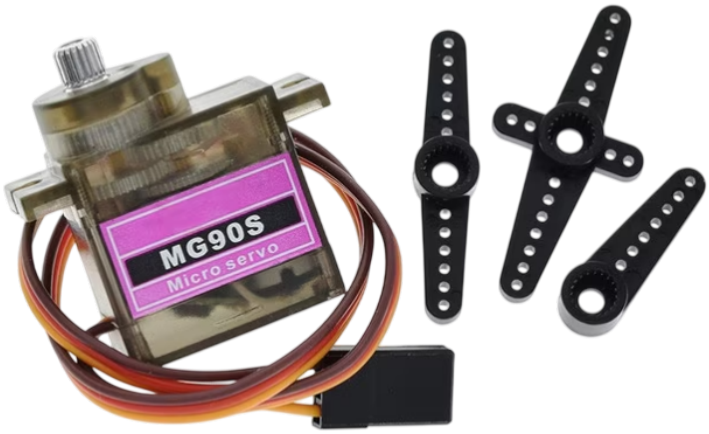


Figure 1: Picture of the MG90-S servomotor used in the project

2.2 Inertial Measurement Unit: MPU-6050

Spatial attitude estimation was achieved using the **InvenSense MPU-6050**, a micro-electromechanical system (MEMS) Inertial Measurement Unit (IMU) featuring a 3-axis accelerometer and a 3-axis gyroscope on a single silicon die [2]. For this project, the 3-axis accelerometer was heavily utilized to determine the gravitational force vector acting on the gimbal system.

The sensor interfaces with the ATmega328P via the I^2C (Two-Wire Interface) protocol. By default, the MPU-6050 initializes in a low-power sleep state; therefore, the register-level setup routine explicitly writes a logic `0x00` to the Power Management 1 register (`0x6B`) to enable the internal clocks and sensor hardware. Operating at a standard $100\,\text{kHz}$ bus speed, the microcontroller queries the sensor at memory address `0x3B` to retrieve the spatial data.

Because the MPU-6050 utilizes highly precise 16-bit Analog-to-Digital Converters (ADCs), but the I^2C protocol transmits in 8-bit packets, the software sequentially reads six bytes and utilizes bitwise operations to concatenate the high and low bytes back into signed 16-bit integers. These raw acceleration vectors are subsequently processed using the `atan2()` trigonometric function to calculate the absolute pitch and roll angles in degrees. Finally, pre-calibrated static offsets are subtracted from the result to establish a true mathematical zero, thereby mitigating steady-state errors before the data enters the PID control loop.

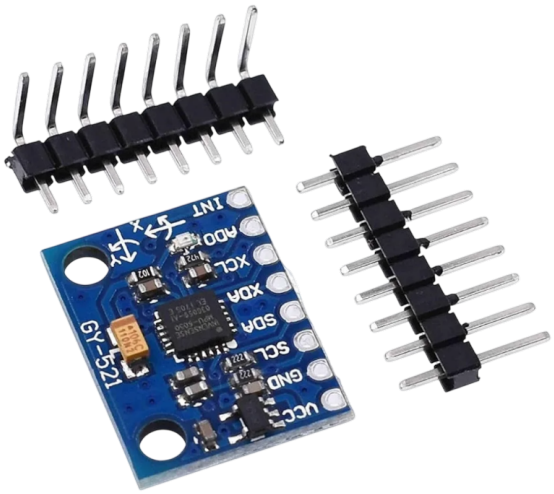


Figure 2: Picture of the MPU6050 inertial sensor used in the project

2.3 Microcontroller: ATmega328P

The central processing unit for the Thrust Vector Control system is the Microchip ATmega328P, an 8-bit AVR RISC-based microcontroller housed on the Arduino Uno development board [3]. Operating at a 16 MHz clock frequency, the microcontroller provides the necessary computational bandwidth to execute floating-point kinematics and PID control algorithms at a highly stable 66 Hz refresh rate.

To maximize timing precision and eliminate the computational overhead of standard abstraction libraries, the firmware was implemented utilizing strictly register-level C programming. Specifically, the 16-bit Timer/Counter1 (`TCCR1A`, `TCCR1B`) was configured for direct Fast PWM generation to drive the actuators, while the internal Two-Wire Interface (TWI) registers were manipulated for non-blocking I^2C communication with the IMU. This low-level hardware control guarantees strictly deterministic execution timing, preventing jitter in the time delta (Δt) that would otherwise destabilize the derivative term of the control loop.

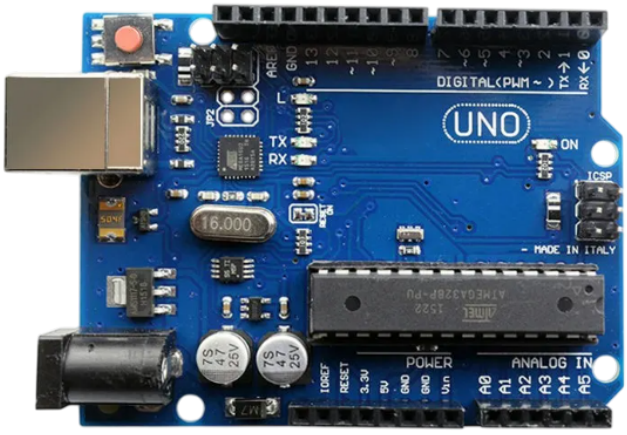


Figure 3: Picture of the Arduino Uno chinese copy board used in the project

2.4 Structural Components: 3D-Printed Gimbal Mount

The mechanical framework of the Thrust Vector Control system relies on custom 3D-printed components designed to house the actuators, sensors, and the main microcontroller.

The two-axis gimbal assembly consists of a concentric inner and outer ring system. The outer ring is directly actuated by the X-axis servo, while the inner ring is controlled by the Y-axis servo. This nested configuration allows for independent pitch and roll articulation of the simulated rocket nozzle (represented by the cylindrical cardboard tube) without the mechanical linkage of one axis binding the other. The servo motors are secured directly to these printed structural frames, ensuring rigid mechanical power transmission and minimizing backlash.

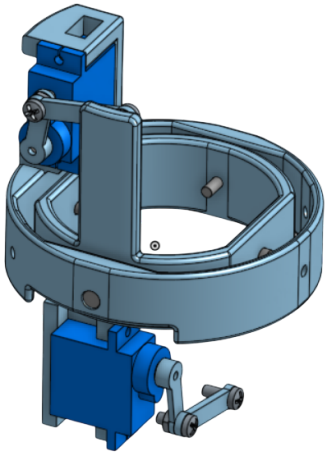


Figure 4: CAD rendering of the two-axis Thrust Vector Control gimbal assembly and servo motor linkages

To isolate the sensitive control electronics from the mechanical actuation, a separate 3D-printed stand was fabricated. This flat mounting plate features integrated standoffs configured to securely anchor both the Arduino Uno and the MPU-6050 IMU. Securing the IMU flat against this rigid, stationary mount is critical; it ensures the calculated spatial attitude accurately reflects the true orientation of the main rocket body and prevents mechanical vibration noise from the servos from corrupting the raw acceleration data.



Figure 5: 3D-printed mounting plate featuring integrated standoffs for the control hardware

3 Description of work

3.1 Proportional-Integral-Derivative (PID) Control Law

To maintain the thrust vector at a perfectly vertical orientation (a 0° setpoint), an active closed-loop feedback system was implemented using a Proportional-Integral-Derivative (PID) control architecture. The controller continuously calculates an error value, $e(t)$, representing the angular deviation between the desired setpoint and the measured spatial attitude.

The standard continuous-time ideal PID algorithm, widely established in classical control theory [4], is defined by the following equation:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$$

Where:

- $u(t)$ is the corrective control signal output.
- K_p, K_i, and K_d are the proportional, integral, and derivative tuning gains, respectively.
- $e(t)$ is the current calculated angular error.

Because the microcontroller operates in discrete time steps rather than continuous time, this control law was discretized for implementation in the digital flight loop. Executing at a strictly governed refresh interval of $\Delta t \approx 15$ ms, the control loop approximates the integral term using a Riemann sum accumulation and calculates the derivative term using a backward finite difference method [4].

The implemented discrete-time control equation is:

$$u[k] = K_p e[k] + K_i \sum_{n=0}^{k} e[n]\Delta t + K_d \frac{e[k] - e[k-1]}{\Delta t}$$

In the context of the physical gimbal, the proportional term (K_p) provides an immediate restorative force linearly proportional to the tilt of the rocket. The integral term (K_i) accumulates persistent microscopic errors over time, successfully correcting steady-state biases such as asymmetrical structural weight or wire tension. Finally, the derivative term (K_d) evaluates the rate of angular change, applying a mathematical damping effect that prevents the actuators from aggressively overshooting the center setpoint during rapid corrections. Two entirely independent PID loops were instanced to evaluate and command the X-axis and Y-axis gimbals simultaneously without cross-axis interference.

3.2 System Limitations and Mechanical Challenges

The transition from theoretical control logic to a physical hardware prototype introduced several mechanical constraints and manufacturing challenges. Successfully mitigating these physical limitations was a primary focus during the calibration and testing phases of the project.

3.2.1 Manufacturing Tolerances and Actuation Limits

The structural components of the gimbal were fabricated utilizing rapid prototyping (3D printing). Due to inherent dimensional inaccuracies and variations in the 3D printing process, the assembled gimbal experienced mechanical interference at the extremes of its travel paths. While the mathematical model and the MG90S servos are capable of a full 180° rotation, physical collisions between the inner and outer gimbal rings prevented this maximum sweep.

To prevent the servo motors from stalling and drawing excessive current against these physical barriers, the theoretical operational envelope was mapped, and strict travel limits were established. These limits (e.g., restricting the X-axis to a range of 100° to 180°) were successfully implemented in the register-level C code, ensuring the control system never commanded an angle outside the safe mechanical bounds of the printed structure.

3.2.2 Linkage Failure and Field Modification

During the iterative PID tuning phase, the system experienced a mechanical fatigue failure. The rapid, high-frequency torque reversals commanded by the derivative term of the controller caused one of the stock plastic servo linkages to fracture.

To restore continuous actuation to the axis without delaying the testing schedule, a field-expedient mechanical modification was implemented(aka a DIY fix). A high-tensile steel wire (repurposed standard office paperclip) was manually formed into a custom hook linkage to replace the fractured plastic arm on the Y axis.

While this asymmetrical linkage introduced a slight degree of mechanical slop and hysteresis into the physical gimbal, it provided an excellent real-world test for the closed-loop control system. The integral term (K_i) of the PID controller successfully identified this new mechanical bias and continuously accumulated the necessary error to compensate for the slack in the wire hook, perfectly maintaining the stability of the 0° vertical setpoint despite the compromised hardware.

3.3 Electrical Schematic and Wiring Connections

The electrical architecture of the Thrust Vector Control system was designed for minimal latency, establishing direct data and actuation pathways between the ATmega328P and the peripherals.

The MPU-6050 IMU communicates via the I^2C protocol. Consequently, the sensor's Serial Clock (SCL) and Serial Data (SDA) lines are hardwired to the microcontroller's dedicated hardware TWI pins (Analog 5 and Analog 4, respectively). Pull-up resistors for the I^2C bus are integrated internally on the GY-521 breakout board, eliminating the need for external passive components.

Actuation signals for the MG90S servos are driven by the hardware Timer/Counter1. To physically link the servos to the Fast PWM outputs defined in the register-level firmware, the X-axis servo signal wire is connected to digital pin PD3 (Timer1 Compare Match A), and the Y-axis servo is connected to digital pin PD5 (Timer1 Compare Match B).

To ensure system stability, power distribution is divided into two isolated rails. The logic-level MPU-6050 is powered directly from the Arduino's onboard 5 V voltage regulator. However, driving inductive loads such as servo motors from the microcontroller can cause voltage sags or brown-outs during rapid torque reversals. Therefore, the MG90S actuators are driven by an independent, external 5 V power supply. To ensure a valid reference voltage for the PWM control signals, the ground terminal of the external power supply is tied directly to the ATmega328P to establish a common ground rail.

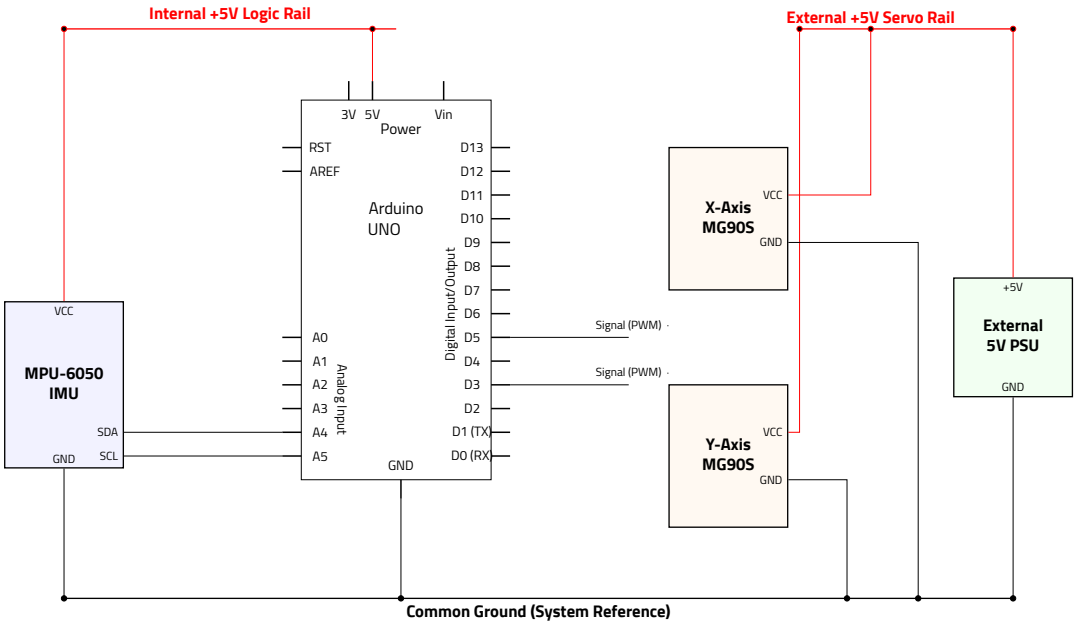


Figure 6: Complete wiring schematic of the system

3.4 Setup architecture

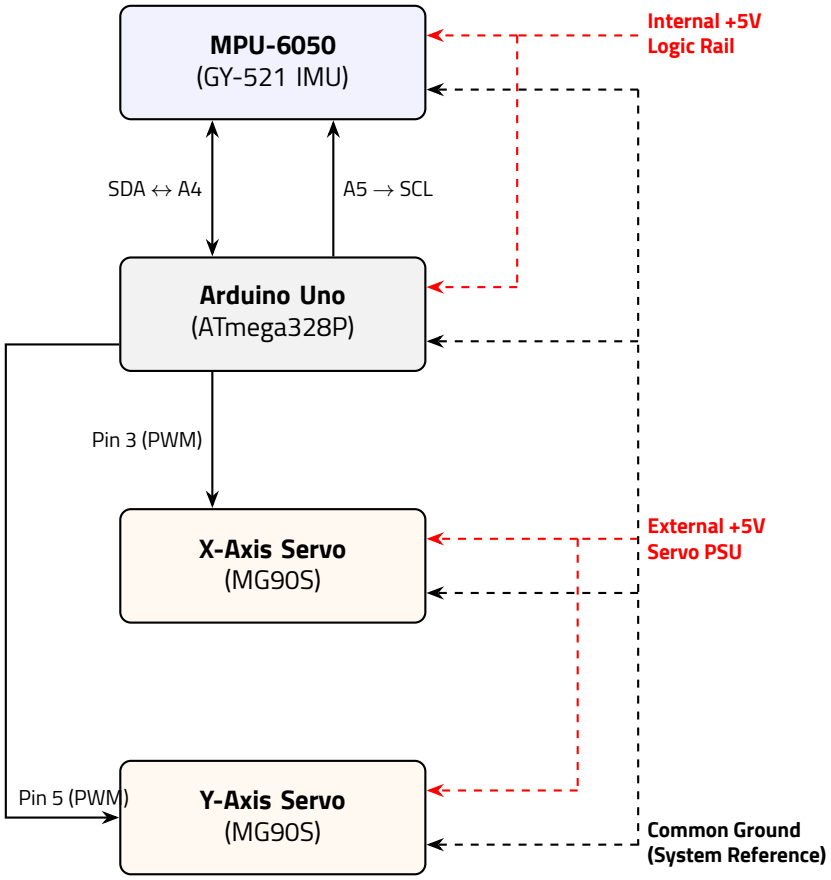


Figure 7: System architecture diagram detailing control and supply pathways

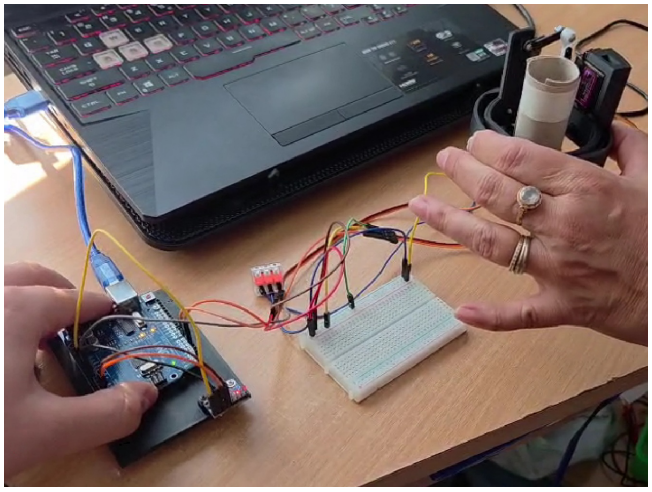


Figure 8: Photograph of the system while working (my mom was holding the gimbal rings)

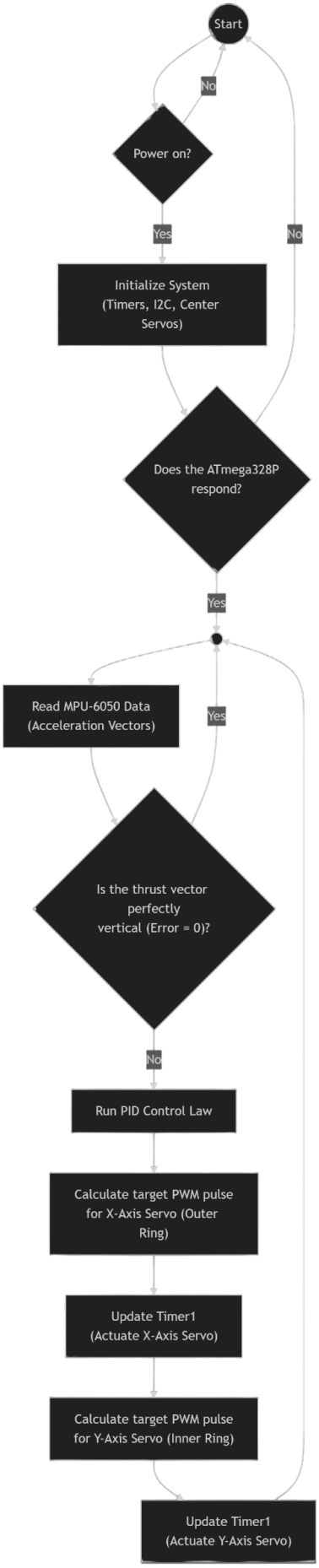


Figure 9: System flowchart illustrating the control logic of the system

3.5 Pinout Table

PIN	Functionality	Description	Component
5V	Power Output	Internal 5V Logic Rail	MPU-6050 IMU
GND	Ground	Sensor Ground	MPU-6050 IMU
GND	Ground	Common Ground (System Reference)	External PSU & Servos
A5	SCL	I2C communication	MPU-6050 IMU
A4	SDA	I2C communication	MPU-6050 IMU
D3	Digital Output	Fast PWM (Timer1)	X-Axis Servo (MG90S)
D5	Digital Output	Fast PWM (Timer1)	Y-Axis Servo (MG90S)

Table 1: Arduino pins used in the Thrust Vector Control project and their functionality.

4 Working with registers

4.1 General Purpose Input/Output (GPIO) Registers

To actuate the physical gimbal mechanism, the microcontroller must output the calculated PWM signals to the servo motors. The Data Direction Register for port D (`DDRD`) dictates whether the pins on this port act as inputs or outputs. Because the X-axis and Y-axis servos are physically wired to digital pins `PD3` and `PD5` respectively, these specific pins must be configured as outputs by setting bits 3 and 5 to logic 1.

```
DDRD |= (1 << PD3) | (1 << PD5);
```

4.2 Timer0 Configuration: System Timing

Because the standard Arduino `millis()` function relies on background interrupts that can interfere with time-critical control loops, a custom hardware timer was implemented using the 8-bit `TIMER0`. This timer generates an interrupt exactly every 1 millisecond to establish a highly deterministic time delta (dt) for the derivative part of the PID controller.

`TIMER0` was configured in Clear Timer on Compare Match (CTC) mode by setting the `WGM01` bit in the `TCCR0A` register.

```
TCCR0A = (1 << WGM01);
```

To achieve the 1 ms interrupt interval from the 16 MHz system clock, the prescaler was set to 64 by configuring the `CS01` and `CS00` bits in the `TCCR0B` register.

```
TCCR0B = (1 << CS01) | (1 << CS00);
```

With a prescaler of 64, the timer clock period is calculated as:

$$T_{clk} = \frac{1}{\frac{16\times10^6}{64}} = 4\mu s$$

To trigger an interrupt every 1000 μs (1 ms), the Output Compare Register (`OCR0A`) must be set to the required number of ticks. Since the counter is zero-indexed, the value is calculated as:

$$OCR0A = \frac{1000\mu s}{4\mu s} - 1 = 249$$

```
OCR0A = 249;
TIMSK0 = (1 << OCIE0A);
```

The `TIMSK0` register enables the interrupt on compare match A, triggering the ISR that increments the `timer0_millis` volatile variable.

4.3 Timer1 Configuration: Fast PWM Generation

The MG90S servo motors require a 50 Hz PWM signal (a 20 ms frame) where the pulse width dictates the physical angle. To eliminate the overhead of the standard <Servo.h> library, the 16-bit TIMER1 was utilized to generate direct hardware PWM signals.

The timer was configured for Fast PWM mode with ICR1 defining the TOP value. This required setting the WGM11, WGM12, and WGM13 bits. The prescaler was set to 8 via the CS11 bit.

```
TCCR1A = (1 << WGM11);
TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS11);
```

With a prescaler of 8, the timer ticks every 0.5 μs. To establish the 20 ms refresh rate (50 Hz), the Input Capture Register 1 (ICR1) was set to 39999 ($40000 \times 0.5\mu s = 20000\mu s$).

```
ICR1 = 39999;
TIMSK1 = (1 << OCIE1A) | (1 << OCIE1B) | (1 << TOIE1);
```

4.3.1 Duty Cycle Calculus for Actuation

To rotate the servos from 0 to 180 degrees, the MG90S requires a pulse width ranging from 544 μs to 2400 μs. Because TIMER1 ticks every 0.5 μs, the corresponding register values are:

- **0 Degrees:** $\frac{544\mu s}{0.5\mu s} = 1088$ ticks.
- **180 Degrees:** $\frac{2400\mu s}{0.5\mu s} = 4800$ ticks.

The difference between the maximum and minimum tick values is $4800 - 1088 = 3712$. Therefore, to dynamically map a calculated PID angle (0-180) directly to the OCR1A (X-Axis) and OCR1B (Y-Axis) registers, the following linear transformation was written:

```
OCR1A = 1088 + ((long)angle * 3712) / 180;
```

4.4 Two-Wire Interface (TWI) Configuration

To retrieve the raw acceleration vectors from the MPU-6050 IMU, the internal Two-Wire Interface (I^2C protocol) hardware of the ATmega328P was manually configured.

To set the standard 100 kHz I^2C clock speed, the Two Wire Bit Rate Register (TWBR) was set to 72, and the prescaler bits in the Status Register (TWSR) were zeroed out to default to a prescaler of 1. Finally, the TWEN bit was set in the Control Register (TWCR) to enable the interface.

```
TWSR = 0x00;
TWBR = 72;
TWCR = (1 << TWEN);
```

To request and transmit data, direct bitwise manipulation was used on TWCR. For example, initiating a start condition requires setting the TWSTA (Start), TWEN (Enable), and TWINT (Interrupt Flag Clear) bits.

```
TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
while (!(TWCR & (1 << TWINT)));
```

The while loop holds microcontroller execution until the hardware explicitly confirms the pulse has finished by setting the TWINT flag, guaranteeing synchronization between the MCU and the IMU.

5 Conclusions

The objective of this project was to design and implement a functional two-axis Thrust Vector Control (TVC) prototype, bridging theoretical control mathematics with low-level programming. By integrating a custom 3D-printed gimbal with an ATmega328P and an MPU-6050 IMU, the system successfully demonstrated active aerospace stabilization modeling. Utilizing register-level C programming, the firmware achieved highly deterministic execution timing, establishing a stable 66 Hz control loop and precise hardware-driven PWM signals. Despite physical manufacturing constraints and a field-expedient mechanical repair, the robust Proportional-Integral-Derivative (PID) controller continuously accumulated and compensated for hardware biases, effectively maintaining a stable, vertical 0° setpoint and proving the overall viability of the sensor-driven, closed-loop design.

References

[1] TowerPro, MG90S Micro Servo Datasheet, Accessed: 2026-05-24, TowerPro Pte Ltd, 2014. [Online]. Available: https://www.towerpro.com.tw

[2] InvenSense Inc., MPU-6050 Six-Axis (Gyro + Accelerometer) MEMS MotionTracking Device Product Specification, Accessed: 2026-05-24, InvenSense Inc., 2013. [Online]. Available: https://invensense.tdk.com

[3] Microchip Technology Inc., ATmega328P 8-bit AVR Microcontroller Datasheet, Accessed: 2026-05-24, Microchip Technology Inc., 2015. [Online]. Available: https://www.microchip.com

[4] K. Ogata, Modern Control Engineering, 5th. Prentice Hall, 2010, ISBN: 978-0136156734.